

My Project

Generated by Doxygen 1.13.2

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 Vector< T > Class Template Reference	5
3.1.1 Detailed Description	8
3.1.2 Member Typedef Documentation	9
3.1.2.1 const_iterator	9
3.1.2.2 const_reference	9
3.1.2.3 const_reverse_iterator	9
3.1.2.4 iterator	9
3.1.2.5 reference	9
3.1.2.6 reverse_iterator	9
3.1.2.7 size_type	10
3.1.3 Constructor & Destructor Documentation	10
3.1.3.1 Vector() [1/6]	10
3.1.3.2 Vector() [2/6]	10
3.1.3.3 Vector() [3/6]	10
3.1.3.4 ~Vector()	11
3.1.3.5 Vector() [4/6]	11
3.1.3.6 Vector() [5/6]	11
3.1.3.7 Vector() [6/6]	11
3.1.4 Member Function Documentation	11
3.1.4.1 assign() [1/3]	11
3.1.4.2 assign() [2/3]	12
3.1.4.3 assign() [3/3]	12
3.1.4.4 at() [1/2]	12
3.1.4.5 at() [2/2]	12
3.1.4.6 back() [1/2]	13
3.1.4.7 back() [2/2]	13
3.1.4.8 begin() [1/2]	13
3.1.4.9 begin() [2/2]	13
3.1.4.10 capacity()	13
3.1.4.11 cbegin()	14
3.1.4.12 cend()	14
3.1.4.13 clear()	14
3.1.4.14 crbegin()	14
3.1.4.15 crend()	14
3.1.4.16 data() [1/2]	14
3.1.4.17 data() [2/2]	15

3.1.4.18 <code>emplace()</code>	15
3.1.4.19 <code>emplace_back()</code>	15
3.1.4.20 <code>empty()</code>	16
3.1.4.21 <code>end()</code> [1/2]	16
3.1.4.22 <code>end()</code> [2/2]	16
3.1.4.23 <code>erase()</code> [1/4]	16
3.1.4.24 <code>erase()</code> [2/4]	16
3.1.4.25 <code>erase()</code> [3/4]	16
3.1.4.26 <code>erase()</code> [4/4]	17
3.1.4.27 <code>front()</code> [1/2]	17
3.1.4.28 <code>front()</code> [2/2]	17
3.1.4.29 <code>insert()</code> [1/5]	17
3.1.4.30 <code>insert()</code> [2/5]	17
3.1.4.31 <code>insert()</code> [3/5]	18
3.1.4.32 <code>insert()</code> [4/5]	18
3.1.4.33 <code>insert()</code> [5/5]	18
3.1.4.34 <code>max_size()</code>	18
3.1.4.35 <code>operator!=(())</code>	18
3.1.4.36 <code>operator<()</code>	18
3.1.4.37 <code>operator<=()</code>	19
3.1.4.38 <code>operator=()</code> [1/3]	19
3.1.4.39 <code>operator=()</code> [2/3]	19
3.1.4.40 <code>operator=()</code> [3/3]	19
3.1.4.41 <code>operator==(())</code>	19
3.1.4.42 <code>operator>()</code>	19
3.1.4.43 <code>operator>=()</code>	20
3.1.4.44 <code>operator[]()</code> [1/2]	20
3.1.4.45 <code>operator[]()</code> [2/2]	20
3.1.4.46 <code>pop_back()</code>	20
3.1.4.47 <code>push_back()</code> [1/2]	20
3.1.4.48 <code>push_back()</code> [2/2]	20
3.1.4.49 <code>rbegin()</code> [1/2]	21
3.1.4.50 <code>rbegin()</code> [2/2]	21
3.1.4.51 <code>rend()</code> [1/2]	21
3.1.4.52 <code>rend()</code> [2/2]	21
3.1.4.53 <code>reserve()</code>	21
3.1.4.54 <code>resize()</code> [1/2]	21
3.1.4.55 <code>resize()</code> [2/2]	22
3.1.4.56 <code>shrink_to_fit()</code>	22
3.1.4.57 <code>size()</code>	22
3.1.4.58 <code>swap()</code>	22

4 File Documentation	23
4.1 vector.h File Reference	23
4.1.1 Detailed Description	23
4.2 vector.h	24
4.3 vector_capacity.h File Reference	26
4.4 vector_capacity.h	26
4.5 vector_elementAccess.h File Reference	27
4.6 vector_elementAccess.h	27
4.7 Vector_impl.h File Reference	28
4.8 Vector_impl.h	28
4.9 vector_library.h File Reference	29
4.10 vector_library.h	29
4.11 vector_memberFunctions.h File Reference	29
4.12 vector_memberFunctions.h	30
4.13 vector_modifiers.h File Reference	31
4.14 vector_modifiers.h	31
4.15 vector_non_member_functions.h File Reference	36
4.15.1 Function Documentation	36
4.15.1.1 operator<<()	36
4.16 vector_non_member_functions.h	36
Index	39

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Vector< T >	Contiguous grow-able array owning its storage (similar to <code>std::vector</code>)	5
-----------------------------------	--	---

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

vector.h	A light-weight replacement for <code>std::vector</code> that allocates with raw <code>new/delete</code> instead of an allocator. overloads	23
vector_capacity.h		26
vector_elementAccess.h		27
Vector_impl.h		28
vector_library.h		29
vector_memberFunctions.h		29
vector_modifiers.h		31
vector_non_member_functions.h		36

Chapter 3

Class Documentation

3.1 `Vector< T >` Class Template Reference

Contiguous grow-able array owning its storage (similar to `std::vector`).

```
#include <vector.h>
```

Public Types

- using `size_type` = `std::size_t`
Unsigned size type.
- using `reference` = `T&`
Mutable reference.
- using `const_reference` = `const T&`
Immutable reference.

Iterator types

- using `iterator` = `T*`
Pointer iterator.
- using `const_iterator` = `const T*`
Immutable pointer iterator.
- using `reverse_iterator` = `std::reverse_iterator<iterator>`
Reverse iterator.
- using `const_reverse_iterator` = `std::reverse_iterator<const_iterator>`
Const reverse iterator.

Public Member Functions

- [Vector](#) ()
Construct an empty vector.
- [Vector](#) ([size_type](#) count, const T &value=T())
Construct `count` copies of `value`.
- template<typename InputIt, typename = std::enable_if_t<!std::is_integral<InputIt>::value>>
[Vector](#) (InputIt first, InputIt last)
Range constructor.
- [~Vector](#) ()
Destroy the vector and its elements.
- void [assign](#) ([size_type](#) count, const T &value)
Replace contents with `count` copies of `value`. @complexity $O(count)$.
- template<class InputIt, typename = std::enable_if_t<!std::is_integral<InputIt>::value>>
void [assign](#) (InputIt first, InputIt last)
Replace contents with the range `[first, last)`.
- void [assign](#) (std::initializer_list< T > ilist)
Replace contents with an initializer list.
- bool [operator==](#) (const [Vector](#) &other) const
Equality comparison.
- bool [operator!=](#) (const [Vector](#) &other) const
Inequality comparison.
- bool [operator<](#) (const [Vector](#) &other) const
Lexicographical less-than.
- bool [operator<=](#) (const [Vector](#) &other) const
Less-than-or-equal.
- bool [operator>](#) (const [Vector](#) &other) const
Greater-than.
- bool [operator>=](#) (const [Vector](#) &other) const
Greater-than-or-equal.
- [reference](#) at ([size_type](#) pos)
Bounds-checked element access.
- [const_reference](#) at ([size_type](#) pos) const
Bounds-checked element access.
- [reference](#) front ()
Access the first element.
- [const_reference](#) front () const
Access the first element.
- [reference](#) back ()
Access the last element.
- [const_reference](#) back () const
Access the last element.
- constexpr T * [data](#) () noexcept
Direct pointer to the underlying array.
- constexpr const T * [data](#) () const noexcept
Direct pointer to the underlying array.
- constexpr bool [empty](#) () const noexcept
Checks whether the container is empty.
- constexpr [size_type](#) [size](#) () const noexcept
Returns the number of elements.
- constexpr [size_type](#) [max_size](#) () const noexcept

- Maximum number of elements the container could theoretically hold.

 - void `reserve (size_type new_cap)`
Request a capacity at least `new_cap`.
 - constexpr `size_type capacity ()` const noexcept
Returns the number of elements that can be held without reallocation.
 - void `shrink_to_fit ()`
Reduces capacity to fit `size ()`. No effect on size.
 - void `clear ()` noexcept
Erase all elements (`size ()` becomes 0).
 - template<class... Args>
`iterator emplace (const_iterator pos, Args &&... args)`
Construct element in-place before `pos`.
 - template<class... Args>
`reference emplace_back (Args &&... args)`
Construct element in-place at the end.
 - void `pop_back ()`
Remove the last element. Undefined behavior if empty.
 - void `swap (Vector &other)` noexcept
Swap contents with `other` in constant time.

Copy / move constructors

- `Vector (const Vector &other)`
Copy constructor.
- `Vector (Vector &&other)` noexcept
Move constructor (guaranteed nothrow).
- `Vector (std::initializer_list< T > ilist)`
Construct from an initializer list.

Assignment operators

- `Vector & operator= (std::initializer_list< T > ilist)`
Replace contents with an initializer list.
- `Vector & operator= (const Vector &other)`
Copy assignment.
- `Vector & operator= (Vector &&other)` noexcept
Move assignment (guaranteed nothrow).

Unchecked element access

- `T & operator[] (size_type idx)`
Unchecked access (undefined behavior if out of bounds).
- `const T & operator[] (size_type idx) const`
Unchecked const access.

Forward iterators

- constexpr `iterator begin ()` noexcept
Iterator to the first element.
- constexpr `const_iterator begin ()` const noexcept
Iterator to the first element.
- constexpr `const_iterator cbegin ()` const noexcept
Const iterator to the first element.
- constexpr `iterator end ()` noexcept

- *Iterator past the last element.*
- constexpr `const_iterator end` () const noexcept
Iterator past the last element.
- constexpr `const_iterator cend` () const noexcept
Const iterator past the last element.

Reverse iterators

- constexpr `reverse_iterator rbegin` () noexcept
- constexpr `const_reverse_iterator rbegin` () const noexcept
- constexpr `const_reverse_iterator crbegin` () const noexcept
- constexpr `reverse_iterator rend` () noexcept
- constexpr `const_reverse_iterator rend` () const noexcept
- constexpr `const_reverse_iterator crend` () const noexcept

insert

- `iterator insert` (`const_iterator` pos, const T &value)
Insert copy of value before pos.
- `iterator insert` (`const_iterator` pos, T &&value)
Move value before pos.
- `iterator insert` (`const_iterator` pos, `size_type` count, const T &value)
Insert count copies of value before pos.
- template<class InputIt, typename = std::enable_if_t<!std::is_integral<InputIt>::value>>>
`iterator insert` (`const_iterator` pos, InputIt first, InputIt last)
Insert elements from range [first, last) before pos.
- `iterator insert` (`const_iterator` pos, std::initializer_list< T > ilist)
Insert an initializer list before pos.

erase

- `iterator erase` (`iterator` pos)
Erase element at pos. Returns iterator following erased element.
- `iterator erase` (`const_iterator` pos)
Erase element at pos. Returns iterator following erased element.
- `iterator erase` (`iterator` first, `iterator` last)
Erase range [first, last) (half-open).
- `iterator erase` (`const_iterator` first, `const_iterator` last)
Erase range [first, last) (half-open).

push_back

- void `push_back` (const T &value)
Append a copy of value to the end.
- void `push_back` (T &&value)
Append value to the end via move.

resize

- void `resize` (`size_type` count)
Resize to count, value-initializing new elements.
- void `resize` (`size_type` count, const T &value)
Resize to count, fill new elements with value.

3.1.1 Detailed Description

```
template<typename T>
class Vector< T >
```

Contiguous grow-able array owning its storage (similar to `std::vector`).

Template Parameters

<i>T</i>	Element type stored by the container.
----------	---------------------------------------

3.1.2 Member Typedef Documentation

3.1.2.1 const_iterator

```
template<typename T>  
using Vector< T >::const_iterator = const T*
```

Immutable pointer iterator.

3.1.2.2 const_reference

```
template<typename T>  
using Vector< T >::const_reference = const T&
```

Immutable reference.

3.1.2.3 const_reverse_iterator

```
template<typename T>  
using Vector< T >::const_reverse_iterator = std::reverse_iterator<const_iterator>
```

Const reverse iterator.

3.1.2.4 iterator

```
template<typename T>  
using Vector< T >::iterator = T*
```

Pointer iterator.

3.1.2.5 reference

```
template<typename T>  
using Vector< T >::reference = T&
```

Mutable reference.

3.1.2.6 reverse_iterator

```
template<typename T>  
using Vector< T >::reverse_iterator = std::reverse_iterator<iterator>
```

Reverse iterator.

3.1.2.7 size_type

```
template<typename T>
using Vector< T >::size_type = std::size_t
```

Unsigned size type.

3.1.3 Constructor & Destructor Documentation

3.1.3.1 Vector() [1/6]

```
template<typename T>
Vector< T >::Vector () [inline]
```

Construct an empty vector.

No allocation is performed.

Postcondition

```
size() == 0 && capacity() == 0.
```

3.1.3.2 Vector() [2/6]

```
template<typename T>
Vector< T >::Vector (
    size_type count,
    const T & value = T()) [inline]
```

Construct count copies of value.

Equivalent to default construction followed by `assign(count, value)`.

Parameters

<i>count</i>	Number of elements to create.
<i>value</i>	Value to copy into each new element. @complexity O(count).

3.1.3.3 Vector() [3/6]

```
template<typename T>
template<typename InputIt, typename = std::enable_if_t<!std::is_integral<InputIt>::value>>
Vector< T >::Vector (
    InputIt first,
    InputIt last) [inline]
```

Range constructor.

Constructs a vector with the contents of the iterator range `[first, last)`.

Template Parameters

<i>InputIt</i>	Non-integral input iterator type.
----------------	-----------------------------------

Parameters

<i>first</i>	Iterator to the first element.
<i>last</i>	Sentinel past the last element. @complexity O(distance(first, last)).

3.1.3.4 ~Vector()

```
template<typename T>
Vector< T >::~~Vector () [inline]
```

Destroy the vector and its elements.

Invokes the destructor on each element in reverse order of construction and releases the allocated buffer. @complexity O(size()).

3.1.3.5 Vector() [4/6]

```
template<typename T>
Vector< T >::Vector (
    const Vector< T > & other)
```

Copy constructor.

3.1.3.6 Vector() [5/6]

```
template<typename T>
Vector< T >::Vector (
    Vector< T > && other) [noexcept]
```

Move constructor (guaranteed nothrow).

3.1.3.7 Vector() [6/6]

```
template<typename T>
Vector< T >::Vector (
    std::initializer_list< T > ilist)
```

Construct from an initializer list.

3.1.4 Member Function Documentation

3.1.4.1 assign() [1/3]

```
template<typename T>
template<typename InputIt, typename>
void Vector< T >::assign (
    InputIt first,
    InputIt last)
```

Replace contents with the range [first, last).

Template Parameters

<i>Input↔ It</i>	Non-integral input iterator.
----------------------	------------------------------

Parameters

<i>first</i>	First element in the range.
<i>last</i>	Sentinel past the last element. @complexity O(distance(first, last)).

3.1.4.2 assign() [2/3]

```
template<typename T>
void Vector< T >::assign (
    size_type count,
    const T & value)
```

Replace contents with `count` copies of `value`. @complexity O(count).

3.1.4.3 assign() [3/3]

```
template<typename T>
void Vector< T >::assign (
    std::initializer_list< T > ilist)
```

Replace contents with an initializer list.

3.1.4.4 at() [1/2]

```
template<typename T>
Vector< T >::reference Vector< T >::at (
    size_type pos)
```

Bounds-checked element access.

Parameters

<i>pos</i>	Index of element to return.
------------	-----------------------------

Exceptions

<code>std::out_of_range</code>	If <code>pos >= size()</code> .
--------------------------------	------------------------------------

3.1.4.5 at() [2/2]

```
template<typename T>
Vector< T >::const_reference Vector< T >::at (
    size_type pos) const
```

Bounds-checked element access.

Parameters

<i>pos</i>	Index of element to return.
------------	-----------------------------

Exceptions

<code>std::out_of_range</code>	If <code>pos >= size()</code> .
--------------------------------	------------------------------------

3.1.4.6 back() [1/2]

```
template<typename T>
Vector< T >::reference Vector< T >::back ()
```

Access the last element.

3.1.4.7 back() [2/2]

```
template<typename T>
Vector< T >::const_reference Vector< T >::back () const
```

Access the last element.

3.1.4.8 begin() [1/2]

```
template<typename T>
const_iterator Vector< T >::begin () const [inline], [constexpr], [noexcept]
```

Iterator to the first element.

3.1.4.9 begin() [2/2]

```
template<typename T>
iterator Vector< T >::begin () [inline], [constexpr], [noexcept]
```

Iterator to the first element.

3.1.4.10 capacity()

```
template<typename T>
Vector< T >::size_type Vector< T >::capacity () const [constexpr], [noexcept]
```

Returns the number of elements that can be held without reallocation.

3.1.4.11 cbegin()

```
template<typename T>
const_iterator Vector< T >::cbegin () const [inline], [constexpr], [noexcept]
```

Const iterator to the first element.

3.1.4.12 cend()

```
template<typename T>
const_iterator Vector< T >::cend () const [inline], [constexpr], [noexcept]
```

Const iterator past the last element.

3.1.4.13 clear()

```
template<typename T>
void Vector< T >::clear () [noexcept]
```

Erase all elements (`size()` becomes 0).

3.1.4.14 crbegin()

```
template<typename T>
const_reverse_iterator Vector< T >::crbegin () const [inline], [constexpr], [noexcept]
```

3.1.4.15 crend()

```
template<typename T>
const_reverse_iterator Vector< T >::crend () const [inline], [constexpr], [noexcept]
```

3.1.4.16 data() [1/2]

```
template<typename T>
const T * Vector< T >::data () const [inline], [constexpr], [noexcept]
```

Direct pointer to the underlying array.

Returns

Pointer to the first element (may be `nullptr` when empty).

3.1.4.17 data() [2/2]

```
template<typename T>
T * Vector< T >::data () [inline], [constexpr], [noexcept]
```

Direct pointer to the underlying array.

Returns

Pointer to the first element (may be `nullptr` when empty).

3.1.4.18 emplace()

```
template<typename T>
template<class... Args>
Vector< T >::iterator Vector< T >::emplace (
    const_iterator pos,
    Args &&... args)
```

Construct element in-place before `pos`.

Template Parameters

<i>Args</i>	Constructor argument pack for T.
-------------	----------------------------------

Returns

Iterator pointing to the inserted element.

3.1.4.19 emplace_back()

```
template<typename T>
template<class... Args>
Vector< T >::reference Vector< T >::emplace_back (
    Args &&... args)
```

Construct element in-place at the end.

Template Parameters

<i>Args</i>	Constructor argument pack for T.
-------------	----------------------------------

Returns

Reference to the newly inserted element.

3.1.4.20 empty()

```
template<typename T>
bool Vector< T >::empty () const [constexpr], [noexcept]
```

Checks whether the container is empty.

3.1.4.21 end() [1/2]

```
template<typename T>
const_iterator Vector< T >::end () const [inline], [constexpr], [noexcept]
```

Iterator past the last element.

3.1.4.22 end() [2/2]

```
template<typename T>
iterator Vector< T >::end () [inline], [constexpr], [noexcept]
```

Iterator past the last element.

3.1.4.23 erase() [1/4]

```
template<typename T>
Vector< T >::iterator Vector< T >::erase (
    const_iterator first,
    const_iterator last)
```

Erase range [first, last) (half-open).

3.1.4.24 erase() [2/4]

```
template<typename T>
Vector< T >::iterator Vector< T >::erase (
    const_iterator pos)
```

Erase element at pos. Returns iterator following erased element.

3.1.4.25 erase() [3/4]

```
template<typename T>
Vector< T >::iterator Vector< T >::erase (
    iterator first,
    iterator last)
```

Erase range [first, last) (half-open).

3.1.4.26 erase() [4/4]

```
template<typename T>
Vector< T >::iterator Vector< T >::erase (
    iterator pos)
```

Erase element at `pos`. Returns iterator following erased element.

3.1.4.27 front() [1/2]

```
template<typename T>
Vector< T >::reference Vector< T >::front ()
```

Access the first element.

3.1.4.28 front() [2/2]

```
template<typename T>
Vector< T >::const_reference Vector< T >::front () const
```

Access the first element.

3.1.4.29 insert() [1/5]

```
template<typename T>
Vector< T >::iterator Vector< T >::insert (
    const_iterator pos,
    const T & value)
```

Insert copy of `value` before `pos`.

3.1.4.30 insert() [2/5]

```
template<typename T>
template<class InputIt, typename>
Vector< T >::iterator Vector< T >::insert (
    const_iterator pos,
    InputIt first,
    InputIt last)
```

Insert elements from range `[first, last)` before `pos`.

Template Parameters

<i>InputIt</i>	Non-integral input iterator type.
----------------	-----------------------------------

3.1.4.31 insert() [3/5]

```
template<typename T>
Vector< T >::iterator Vector< T >::insert (
    const_iterator pos,
    size_type count,
    const T & value)
```

Insert count copies of value before pos.

3.1.4.32 insert() [4/5]

```
template<typename T>
Vector< T >::iterator Vector< T >::insert (
    const_iterator pos,
    std::initializer_list< T > ilist)
```

Insert an initializer list before pos.

3.1.4.33 insert() [5/5]

```
template<typename T>
Vector< T >::iterator Vector< T >::insert (
    const_iterator pos,
    T && value)
```

Move value before pos.

3.1.4.34 max_size()

```
template<typename T>
Vector< T >::size_type Vector< T >::max_size () const [constexpr], [noexcept]
```

Maximum number of elements the container could theoretically hold.

3.1.4.35 operator"!="()

```
template<typename T>
bool Vector< T >::operator!= (
    const Vector< T > & other) const
```

Inequality comparison.

3.1.4.36 operator<()

```
template<typename T>
bool Vector< T >::operator< (
    const Vector< T > & other) const
```

Lexicographical less-than.

3.1.4.37 operator<=()

```
template<typename T>
bool Vector< T >::operator<= (
    const Vector< T > & other) const
```

Less-than-or-equal.

3.1.4.38 operator=() [1/3]

```
template<typename T>
Vector< T > & Vector< T >::operator= (
    const Vector< T > & other)
```

Copy assignment.

3.1.4.39 operator=() [2/3]

```
template<typename T>
Vector< T > & Vector< T >::operator= (
    std::initializer_list< T > ilist)
```

Replace contents with an initializer list.

3.1.4.40 operator=() [3/3]

```
template<typename T>
Vector< T > & Vector< T >::operator= (
    Vector< T > && other) [noexcept]
```

Move assignment (guaranteed nothrow).

3.1.4.41 operator==(())

```
template<typename T>
bool Vector< T >::operator==(
    const Vector< T > & other) const
```

Equality comparison.

Returns

true if sizes are equal and all elements compare equal. @complexity O(size()).

3.1.4.42 operator>()

```
template<typename T>
bool Vector< T >::operator> (
    const Vector< T > & other) const
```

Greater-than.

3.1.4.43 operator>=()

```
template<typename T>
bool Vector< T >::operator>= (
    const Vector< T > & other) const
```

Greater-than-or-equal.

3.1.4.44 operator[]() [1/2]

```
template<typename T>
T & Vector< T >::operator[] (
    size_type idx) [inline]
```

Unchecked access (undefined behavior if out of bounds).

3.1.4.45 operator[]() [2/2]

```
template<typename T>
const T & Vector< T >::operator[] (
    size_type idx) const [inline]
```

Unchecked const access.

3.1.4.46 pop_back()

```
template<typename T>
void Vector< T >::pop_back ()
```

Remove the last element. Undefined behavior if empty.

3.1.4.47 push_back() [1/2]

```
template<typename T>
void Vector< T >::push_back (
    const T & value)
```

Append a copy of value to the end.

3.1.4.48 push_back() [2/2]

```
template<typename T>
void Vector< T >::push_back (
    T && value)
```

Append value to the end via move.

3.1.4.49 rbegin() [1/2]

```
template<typename T>
const_reverse_iterator Vector< T >::rbegin () const [inline], [constexpr], [noexcept]
```

3.1.4.50 rbegin() [2/2]

```
template<typename T>
reverse_iterator Vector< T >::rbegin () [inline], [constexpr], [noexcept]
```

3.1.4.51 rend() [1/2]

```
template<typename T>
const_reverse_iterator Vector< T >::rend () const [inline], [constexpr], [noexcept]
```

3.1.4.52 rend() [2/2]

```
template<typename T>
reverse_iterator Vector< T >::rend () [inline], [constexpr], [noexcept]
```

3.1.4.53 reserve()

```
template<typename T>
void Vector< T >::reserve (
    size_type new_cap)
```

Request a capacity at least `new_cap`.

Parameters

<code>new_cap</code>	Desired capacity (elements).
----------------------	------------------------------

Exceptions

<code>std::bad_alloc</code>	If allocation fails.
-----------------------------	----------------------

3.1.4.54 resize() [1/2]

```
template<typename T>
void Vector< T >::resize (
    size_type count)
```

Resize to `count`, value-initializing new elements.

3.1.4.55 `resize()` [2/2]

```
template<typename T>
void Vector< T >::resize (
    size_type count,
    const T & value)
```

Resize to `count`, fill new elements with `value`.

3.1.4.56 `shrink_to_fit()`

```
template<typename T>
void Vector< T >::shrink_to_fit ()
```

Reduces capacity to fit `size()`. No effect on size.

3.1.4.57 `size()`

```
template<typename T>
Vector< T >::size_type Vector< T >::size () const [constexpr], [noexcept]
```

Returns the number of elements.

3.1.4.58 `swap()`

```
template<typename T>
void Vector< T >::swap (
    Vector< T > & other) [noexcept]
```

Swap contents with `other` in constant time.

Parameters

<i>other</i>	Vector to swap with.
--------------	----------------------

Postcondition

This vector contains the previous contents of `other` and vice-versa.

The documentation for this class was generated from the following files:

- [vector.h](#)
- [vector_capacity.h](#)
- [vector_elementAccess.h](#)
- [Vector_impl.h](#)
- [vector_memberFunctions.h](#)
- [vector_modifiers.h](#)
- [vector_non_member_functions.h](#)

Chapter 4

File Documentation

4.1 vector.h File Reference

A light-weight replacement for `std::vector` that allocates with raw `new/delete` instead of an allocator. overloads.

```
#include "vector_library.h"
#include "Vector_impl.h"
#include "vector_modifiers.h"
#include "vector_capacity.h"
#include "vector_elementAccess.h"
#include "vector_memberFunctions.h"
#include "vector_non_member_functions.h"
```

Classes

- class [Vector< T >](#)
Contiguous grow-able array owning its storage (similar to `std::vector`).

4.1.1 Detailed Description

A light-weight replacement for `std::vector` that allocates with raw `new/delete` instead of an allocator. overloads.

The implementation is distributed across several helper headers included at the bottom of this file:

- [Vector_impl.h](#) — core private helpers (growth, moves)
- [vector_memberFunctions.h](#) — constructors/assignments
- [vector_elementAccess.h](#) — element accessors
- [vector_capacity.h](#) — capacity helpers
- [vector_modifiers.h](#) — modifiers

Note

This header **does not** depend on allocators; memory is managed with `::operator new[]` / `↔ ::operator delete[]`.

4.2 vector.h

[Go to the documentation of this file.](#)

```

00001 // Vector.h
00002 #pragma once
00003
00021
00022 #include "vector_library.h"
00023
00030 template<typename T>
00031 class Vector {
00032 private:
00034     T* array = nullptr;
00036     size_t size_ = 0;
00038     size_t capacity_ = 0;
00039
00050     void double_the_capacity();
00051
00052 public:
00053     //-----
00054     // Member types
00055     //-----
00056     using size_type      = std::size_t;
00057     using reference      = T&;
00058     using const_reference = const T&;
00059
00062     using iterator       = T*;
00063     using const_iterator = const T*;
00064     using reverse_iterator = std::reverse_iterator<iterator>;
00065     using const_reverse_iterator = std::reverse_iterator<const_iterator>;
00066
00067     //-----
00068     // Constructors / Destructor
00069     //-----
00071
00078     Vector() : array(nullptr), size_(0), capacity_(0) {}
00079
00089     Vector(size_type count, const T& value = T())
00090         : array(nullptr), size_(0), capacity_(0) {
00091         assign(count, value);
00092     }
00093
00105     template<typename InputIt, typename = std::enable_if_t<!std::is_integral<InputIt>::value>
00106     Vector(InputIt first, InputIt last)
00107         : array(nullptr), size_(0), capacity_(0) {
00108         for (; first != last; ++first) {
00109             push_back(*first);
00110         }
00111     }
00112
00120     ~Vector() {
00121         for (size_type i = 0; i < size_; ++i) {
00122             array[i].~T();
00123         }
00124         ::operator delete[](array);
00125     }
00126
00127     //-----
00128     // Copy / move semantics (defined in vector_memberFunctions.h)
00129     //-----
00130
00133
00134     Vector(const Vector& other);
00136     Vector(Vector&& other) noexcept;
00138     Vector(std::initializer_list<T> ilist);
00140
00143
00144     Vector& operator=(std::initializer_list<T> ilist);
00146     Vector& operator=(const Vector& other);
00148     Vector& operator=(Vector&& other) noexcept;
00150
00151     //-----
00152     // assign()
00153     //-----
00154
00159     void assign(size_type count, const T& value);
00160
00169     template<class InputIt, typename = std::enable_if_t<!std::is_integral<InputIt>::value>
00170     void assign(InputIt first, InputIt last);
00171
00175     void assign(std::initializer_list<T> ilist);
00176
00177     //-----
00178     // Comparisons
00179     //-----

```

```

00180
00186     bool operator==(const Vector& other) const;
00187
00189     bool operator!=(const Vector& other) const;
00191     bool operator< (const Vector& other) const;
00193     bool operator<= (const Vector& other) const;
00195     bool operator> (const Vector& other) const;
00197     bool operator>= (const Vector& other) const;
00198
00199     //-----
00200     // Element access
00201     //-----
00202
00208     reference at(size_type pos);
00209
00211     const_reference at(size_type pos) const;
00212
00215
00216     T& operator[](size_type idx) { return array[idx]; }
00218     const T& operator[](size_type idx) const { return array[idx]; }
00220
00222     reference front();
00224     const_reference front() const;
00225
00227     reference back();
00229     const_reference back() const;
00230
00235     constexpr T* data() noexcept { return array; }
00236
00238     constexpr const T* data() const noexcept { return array; }
00239
00240     //-----
00241     // Iterators
00242     //-----
00243
00246
00247     constexpr iterator begin() noexcept { return array; }
00249     constexpr const_iterator begin() const noexcept { return array; }
00251     constexpr const_iterator cbegin() const noexcept { return array; }
00252
00254     constexpr iterator end() noexcept { return array + size_; }
00256     constexpr const_iterator end() const noexcept { return array + size_; }
00258     constexpr const_iterator cend() const noexcept { return array + size_; }
00260
00263     constexpr reverse_iterator rbegin() noexcept { return reverse_iterator(end()); }
00264     constexpr const_reverse_iterator rbegin() const noexcept { return const_reverse_iterator(end()); }
00265     constexpr const_reverse_iterator crbegin() const noexcept { return const_reverse_iterator(end()); }
00266 }
00267
00267     constexpr reverse_iterator rend() noexcept { return reverse_iterator(begin()); }
00268     constexpr const_reverse_iterator rend() const noexcept { return const_reverse_iterator(begin()); }
00269     constexpr const_reverse_iterator crend() const noexcept { return const_reverse_iterator(begin()); }
00270 }
00271
00272     //-----
00273     // Capacity
00274     //-----
00275
00277     constexpr bool empty() const noexcept;
00278
00280     constexpr size_type size() const noexcept;
00281
00283     constexpr size_type max_size() const noexcept;
00284
00290     void reserve(size_type new_cap);
00291
00293     constexpr size_type capacity() const noexcept;
00294
00296     void shrink_to_fit();
00297
00298     //-----
00299     // Modifiers
00300     //-----
00301
00303     void clear() noexcept;
00304
00307
00308     iterator insert(const_iterator pos, const T& value);
00310     iterator insert(const_iterator pos, T&& value);
00312     iterator insert(const_iterator pos, size_type count, const T& value);
00313
00318     template<class InputIt, typename = std::enable_if_t<!std::is_integral<InputIt>::value>
00319     iterator insert(const_iterator pos, InputIt first, InputIt last);
00320
00322     iterator insert(const_iterator pos, std::initializer_list<T> ilist);
00324
00330     template<class... Args>

```

```

00331     iterator emplace(const_iterator pos, Args&&... args);
00332
00333
00334     iterator erase(iterator pos);
00335     iterator erase(const_iterator pos);
00336     iterator erase(iterator first, iterator last);
00337     iterator erase(const_iterator first, const_iterator last);
00338
00339
00340     void push_back(const T& value);
00341     void push_back(T&& value);
00342
00343
00344     template<class... Args>
00345     reference emplace_back(Args&&... args);
00346
00347
00348     void pop_back();
00349
00350
00351     void resize(size_type count);
00352     void resize(size_type count, const T& value);
00353
00354
00355     void swap(Vector& other) noexcept;
00356 };
00357
00358 //-----
00359 // Implementation includes
00360 //-----
00361 #include "Vector_impl.h"
00362 #include "vector_modifiers.h"
00363 #include "vector_capacity.h"
00364 #include "vector_elementAccess.h"
00365 #include "vector_memberFunctions.h"
00366 #include "vector_non_member_functions.h"

```

4.3 vector_capacity.h File Reference

```
#include "vector_library.h"
```

4.4 vector_capacity.h

[Go to the documentation of this file.](#)

```

00001 // vector_capacity.h
00002 #pragma once
00003
00004 #include "vector_library.h"
00005
00006 // empty
00007 // checks whether the container is empty
00008 // (public member function)
00009 template<typename T>
00010 constexpr bool Vector<T>::empty() const noexcept {
00011     return size_ == 0;
00012 }
00013
00014 // size
00015 // returns the number of elements
00016 // (public member function)
00017 template<typename T>
00018 constexpr typename Vector<T>::size_type Vector<T>::size() const noexcept {
00019     return size_;
00020 }
00021
00022 // max_size
00023 // returns the maximum possible number of elements
00024 // (public member function)
00025 template<typename T>
00026 constexpr typename Vector<T>::size_type Vector<T>::max_size() const noexcept {
00027     return std::numeric_limits<size_type>::max() / sizeof(T);
00028 }
00029
00030 // reserve
00031 // reserves storage

```



```

00032 // (public member function)
00033 template<typename T>
00034 void Vector<T>::reserve(size_type n) {
00035     if (n <= capacity_)
00036         return;
00037     T* new_array = static_cast<T*>(
00038         ::operator new[](n * sizeof(T))
00039     );
00040
00041     for (size_type i = 0; i < size_; ++i) {
00042         new (&new_array[i]) T(std::move(array[i]));
00043         array[i].~T();
00044     }
00045
00046     ::operator delete[](array);
00047     array = new_array;
00048     capacity_ = n;
00049 }
00050
00051 // capacity
00052 // returns the number of elements that can be held in currently allocated storage
00053 // (public member function)
00054 template<typename T>
00055 constexpr typename Vector<T>::size_type Vector<T>::capacity() const noexcept {
00056     return capacity_;
00057 }
00058
00059 // shrink_to_fit
00060 // reduces memory usage by freeing unused memory
00061 // (public member function)
00062 template<typename T>
00063 void Vector<T>::shrink_to_fit() {
00064     if (capacity_ == size_)
00065         return; // nothing to shrink
00066
00067     T* new_array = static_cast<T*>(
00068         ::operator new[](size_ * sizeof(T))
00069     );
00070
00071     try {
00072         for (size_type i = 0; i < size_; ++i) {
00073             new (&new_array[i]) T(std::move(array[i]));
00074             array[i].~T();
00075         }
00076     } catch (...) {
00077         for (size_type j = 0; j < size_; ++j) {
00078             new_array[j].~T();
00079         }
00080         ::operator delete[](new_array);
00081         throw;
00082     }
00083
00084     ::operator delete[](array);
00085     array = new_array;
00086     capacity_ = size_;
00087 }

```

4.5 vector_elementAccess.h File Reference

```
#include "vector_library.h"
```

4.6 vector_elementAccess.h

[Go to the documentation of this file.](#)

```

00001 // vector_elementAccess.h
00002 #pragma once
00003
00004 #include "vector_library.h"
00005
00006 // ----- at()
00007 template<typename T>

```

```

00008 typename Vector<T>::reference Vector<T>::at(size_type pos) {
00009     if (pos >= size_) {
00010         throw std::out_of_range("Vector::at: index out of range");
00011     }
00012     return array[pos];
00013 }
00014
00015 template<typename T>
00016 typename Vector<T>::const_reference Vector<T>::at(size_type pos) const {
00017     if (pos >= size_) {
00018         throw std::out_of_range("Vector::at: index out of range");
00019     }
00020     return array[pos];
00021 }
00022
00023 // ----- front()
00024 template<typename T>
00025 typename Vector<T>::reference Vector<T>::front() {
00026     if (empty()) {
00027         throw std::out_of_range("Vector::front(): empty vector");
00028     }
00029     return *begin();
00030 }
00031
00032 template<typename T>
00033 typename Vector<T>::const_reference Vector<T>::front() const {
00034     if (empty()) {
00035         throw std::out_of_range("Vector::front(): empty vector");
00036     }
00037     return *begin();
00038 }
00039
00040 // ----- back()
00041 template<typename T>
00042 typename Vector<T>::reference Vector<T>::back() {
00043     if (empty()) {
00044         throw std::out_of_range("Vector::back(): empty vector");
00045     }
00046     return *(end()-1);
00047 }
00048
00049 template<typename T>
00050 typename Vector<T>::const_reference Vector<T>::back() const {
00051     if (empty()) {
00052         throw std::out_of_range("Vector::front(): empty vector");
00053     }
00054     return *(end()-1);
00055 }

```

4.7 Vector_impl.h File Reference

```
#include "vector_library.h"
```

4.8 Vector_impl.h

[Go to the documentation of this file.](#)

```

00001 // vector_impl.h
00002 #pragma once
00003
00004 #include "vector_library.h"
00005
00006 template<typename T>
00007 void Vector<T>::double_the_capacity() {
00008     // Compute new capacity
00009     size_type newCap = std::max<size_type>(size_type(1), capacity_ * 2);
00010
00011     // Allocate uninitialized memory
00012     T* new_array = static_cast<T*>(
00013         ::operator new[](newCap * sizeof(T))
00014     );
00015
00016     // 3) Move-construct existing elements into new_array
00017     size_type i = 0;

```

```

00018     try {
00019         for (; i < size_; ++i) {
00020             new (&new_array[i]) T(std::move(array[i]));
00021         }
00022     } catch (...) {
00023         // Destroys what we did, then frees
00024         for (size_type j = 0; j < i; ++j) {
00025             new_array[j].~T();
00026         }
00027         ::operator delete[](new_array);
00028         throw;
00029     }
00030
00031     // If success --> destroy old objects & free old memory and commit the new one
00032     for (size_type j = 0; j < size_; ++j) {
00033         array[j].~T();
00034     }
00035     ::operator delete[](array);
00036
00037     array = new_array;
00038     capacity_ = newCap;
00039 }

```

4.9 vector_library.h File Reference

```

#include <algorithm>
#include <cassert>
#include <iterator>
#include <limits>
#include <utility>
#include <ranges>
#include <iostream>
#include <sstream>
#include <string>
#include <type_traits>

```

4.10 vector_library.h

[Go to the documentation of this file.](#)

```

00001
00002
00003 #include <algorithm>           // std::move_backward, std::copy
00004 #include <cassert>             // assert
00005 #include <iterator>            // std::distance, iterator traits
00006 #include <limits>              // std::numeric_limits
00007 #include <utility>             // std::move, std::forward, std::swap
00008
00009
00010 #include <ranges>
00011
00012 #include <iostream>
00013 #include <sstream>
00014 #include <string>
00015
00016
00017 #include <type_traits>         // std::enable_if_t, std::is_integral
00018
00019 using std::cout;
00020 using std::endl;
00021 using std::swap;

```

4.11 vector_memberFunctions.h File Reference

```

#include "vector_library.h"

```

4.12 vector_memberFunctions.h

[Go to the documentation of this file.](#)

```

00001 // vector_memberFunctions.h
00002 #pragma once
00003
00004 #include "vector_library.h"
00005
00006 template<typename T>
00007 void Vector<T>::assign(size_type count, const T& value){
00008
00009     T* new_array = static_cast<T*>(
00010         ::operator new[](count * sizeof(T))
00011     );
00012
00013     size_type constructed = 0;
00014     try {
00015         for (; constructed < count; ++constructed) {
00016             new (&new_array[constructed]) T(value);
00017         }
00018     } catch (...) {
00019         // If fails destroys what we did, then frees the memory
00020         for (size_type j = 0; j < constructed; ++j)
00021             new_array[j].~T();
00022         ::operator delete[](new_array);
00023         throw;
00024     }
00025
00026     for (size_type j = 0; j < size_; ++j) {
00027         array[j].~T();
00028     }
00029     ::operator delete[](array);
00030
00031     array = new_array;
00032     size_ = count;
00033     capacity_ = count;
00034 }
00035
00036 template<typename T>
00037 template<typename InputIt, typename>
00038 void Vector<T>::assign(InputIt first, InputIt last){
00039     for (size_type i = 0; i < size_; ++i)
00040         array[i].~T();
00041
00042     size_ = 0;
00043     for (; first != last; ++first)
00044         push_back(*first);
00045 }
00046
00047 template<typename T>
00048 void Vector<T>::assign(std::initializer_list<T> ilist){
00049     assign(ilist.begin(), ilist.end());
00050 }
00051
00052 // ---- Copy constructor
00053 template<typename T>
00054 Vector<T>::Vector(const Vector<T>& other)
00055     : array(other.capacity_
00056         ? static_cast<T*> (::operator new[] (other.capacity_ * sizeof(T)))
00057         : nullptr),
00058       size_(other.size_),
00059       capacity_(other.capacity_))
00060 {
00061     size_type i = 0;
00062     try {
00063         for (; i < other.size_; ++i) {
00064             new (&array[i]) T(other.array[i]);
00065         }
00066     } catch (...) {
00067         // If fails destroys what we did, then frees the memory
00068         for (size_type j = 0; j < i; ++j) {
00069             array[j].~T();
00070         }
00071         ::operator delete[](array);
00072         throw;
00073     }
00074 }
00075
00076
00077 // --- initializer_list
00078 template<typename T>
00079 Vector<T>::Vector(std::initializer_list<T> ilist)
00080     : array(ilist.size()
00081         ? static_cast<T*> (::operator new[] (ilist.size() * sizeof(T)))
00082         : nullptr),

```

```

00083     size_(0),
00084     capacity_(ilist.size())
00085 {
00086     for (auto const &elem : ilist) {
00087         push_back(elem);
00088     }
00089
00090 template<typename T>
00091 Vector<T>& Vector<T>::operator=(std::initializer_list<T> ilist){
00092     assign(ilist);
00093     return *this;
00094 }
00095
00096 // ---- Move constructor
00097 template<typename T>
00098 Vector<T>::Vector(Vector<T>&& other) noexcept
00099     : array(other.array),
00100     size_(other.size_),
00101     capacity_(other.capacity_)
00102     {
00103         other.array    = nullptr;
00104         other.size_    = 0;
00105         other.capacity_ = 0;
00106     }
00107
00108 // ---- Copy assignment operator
00109 template<typename T>
00110 Vector<T>& Vector<T>::operator=(const Vector<T>& other) {
00111     if (this != &other) {
00112         Vector tmp(other);
00113         swap(tmp);
00114     }
00115     return *this;
00116 }
00117
00118 // ---- Move assignment operator
00119 template<typename T>
00120 Vector<T>& Vector<T>::operator=(Vector<T>&& other) noexcept {
00121     if (&other != this){
00122         delete[] array;
00123         array = other.array;
00124         size_ = other.size_;
00125         capacity_ = other.capacity_;
00126
00127         other.array    = nullptr;
00128         other.size_    = 0;
00129         other.capacity_ = 0;
00130     }
00131     return *this;
00132 }

```

4.13 vector_modifiers.h File Reference

```
#include "vector_library.h"
```

4.14 vector_modifiers.h

[Go to the documentation of this file.](#)

```

00001 // vector_modifiers.h
00002 #pragma once
00003
00004 #include "vector_library.h"
00005
00006 // ----- MODIFIERS -----
00007 // clear
00008 // Clears all elements from the vector.
00009 // (public member function)
00010 template <typename T>
00011 void Vector<T>::clear() noexcept
00012 {
00013     for (size_type i = 0; i < size_; ++i)
00014     {
00015         array[i].~T();
00016     }
00017 }

```

```

00016     }
00017     size_ = 0;
00018 }
00019
00020 // insert
00021 // Inserts one or more elements at a given position.
00022 // (public member function)
00023
00024 // 1) Inserts a copy of value before pos.
00025 template <typename T>
00026 typename Vector<T>::iterator Vector<T>::insert(const_iterator pos, const T &value){
00027
00028     size_type idx = pos - begin();
00029
00030     // Make a copy incase it throws
00031     Vector tmp(*this);
00032
00033     while (tmp.size_ + 1 > tmp.capacity_) {
00034         tmp.double_the_capacity();
00035     }
00036
00037     for (size_type i = tmp.size_; i > idx; --i) {
00038         new (&tmp.array[i]) T(std::move(tmp.array[i-1]));
00039         tmp.array[i-1].~T();
00040     }
00041
00042     new (&tmp.array[idx]) T(value);
00043     ++tmp.size_;
00044
00045     // if everything is OK
00046     swap(tmp);
00047
00048     return array + idx;
00049 }
00050
00051 // 2) Inserts value before pos, using move
00052 template <typename T>
00053 typename Vector<T>::iterator Vector<T>::insert(const_iterator pos, T &&value){
00054     size_type idx = pos - begin();
00055
00056     // Make a copy incase it throws
00057     Vector tmp(*this);
00058
00059     while (tmp.size_ + 1 > tmp.capacity_) {
00060         tmp.double_the_capacity();
00061     }
00062
00063     for (size_type i = tmp.size_; i > idx; --i) {
00064         new (&tmp.array[i]) T(std::move(tmp.array[i - 1]));
00065         tmp.array[i - 1].~T();
00066     }
00067
00068     new (&tmp.array[idx]) T(value);
00069     ++tmp.size_;
00070
00071     // if everything is OK
00072     swap(tmp);
00073
00074     return array + idx;
00075 }
00076
00077 // 3) Inserts count copies of the value before pos.
00078 template <typename T>
00079 typename Vector<T>::iterator Vector<T>::insert(const_iterator pos, size_type count, const T &value){
00080
00081     size_type idx = pos - begin();
00082     Vector tmp; // start fresh, not a copy
00083     tmp.reserve(size_ + count);
00084
00085     // 1) copy elements [0, idx)
00086     for (size_type i = 0; i < idx; ++i) {
00087         tmp.push_back(std::move_if_noexcept(array[i]));
00088     }
00089
00090     for (size_type i = 0; i < count; ++i){
00091         tmp.push_back(value);
00092     }
00093
00094     for (size_type i = idx; i < size_; ++i){
00095         tmp.push_back(std::move_if_noexcept(array[i]));
00096     }
00097
00098     // if everything is OK
00099     swap(tmp);
00100
00101     return begin() + idx;
00102 }

```

```

00103
00104 // 4) Inserts elements from range [first, last) before pos.
00105 template<typename T>
00106 template<class InputIt, typename>
00107 typename Vector<T>::iterator Vector<T>::insert(const_iterator pos, InputIt first, InputIt last){
00108     size_type idx = pos - begin();
00109     size_type count = static_cast<size_type>(std::distance(first, last));
00110
00111     if (count == 0)
00112         return const_cast<iterator>(pos);
00113
00114     Vector tmp;
00115     tmp.reserve(size_ + count);
00116
00117     for (size_type i = 0; i < idx; ++i)
00118         tmp.push_back(std::move_if_noexcept(array[i]));
00119
00120     for (; first != last; ++first)
00121         tmp.push_back(*first);
00122
00123     for (size_type i = idx; i < size_; ++i)
00124         tmp.push_back(std::move_if_noexcept(array[i]));
00125
00126     swap(tmp);
00127     return begin() + idx;
00128 }
00129
00130 // 5) Inserts elements from initializer list ilist before pos.
00131 // Equivalent to insert(pos, ilist.begin(), ilist.end()).
00132 template <typename T>
00133 typename Vector<T>::iterator Vector<T>::insert(const_iterator pos, std::initializer_list<T> ilist){
00134     size_type idx = pos - begin();
00135     Vector tmp;
00136     tmp.reserve(size_ + ilist.size());
00137
00138     for (size_type i = 0; i < idx; ++i)
00139         tmp.push_back(std::move_if_noexcept(array[i]));
00140     for (auto &x : ilist)
00141         tmp.push_back(x);
00142     for (size_type i = idx; i < size_; ++i)
00143         tmp.push_back(std::move_if_noexcept(array[i]));
00144
00145     swap(tmp);
00146     return begin() + idx;
00147 }
00148
00149 // 6) emplace (C++11)
00150 // Constructs an element in-place at a given position.
00151 // (public member function)
00152
00153 template<typename T>
00154 template< class... Args >
00155 typename Vector<T>::iterator Vector<T>::emplace(const_iterator pos, Args&&... args){
00156     size_type idx = pos - begin();
00157
00158     if (size_ + 1 > capacity_){
00159         double_the_capacity();
00160     }
00161
00162     for (size_type i = size_; i > idx; --i){
00163         array[i] = std::move(array[i - 1]);
00164     }
00165
00166     // 4) construct the new T in-place
00167     new (&array[idx]) T(std::forward<Args>(args)...);
00168
00169     ++size_;
00170     return array + idx;
00171 }
00172
00173 // 7) erase
00174 // Removes the element at a given position (or a range of elements).
00175 // (public member function)
00176
00177 // singles --->
00178 template <typename T>
00179 typename Vector<T>::iterator Vector<T>::erase(iterator pos)
00180 {
00181     size_type idx = pos - begin();
00182
00183     if (idx == size_ - 1)
00184     {
00185         pop_back();
00186         return end();
00187     }

```

```

00190     }
00191
00192     // Move everything backwards
00193     for (size_type i = idx; i + 1 < size_; ++i)
00194     {
00195         array[i] = std::move(array[i + 1]);
00196     }
00197
00198     pop_back();
00199
00200     return begin() + idx;
00201 }
00202
00203 template <typename T>
00204 typename Vector<T>::iterator
00205 Vector<T>::erase(const_iterator pos)
00206 {
00207     size_type idx = pos - begin();
00208
00209     if (idx == size_ - 1)
00210     {
00211         pop_back();
00212         return end();
00213     }
00214
00215     // Move everything backwards
00216     for (size_type i = idx; i + 1 < size_; ++i)
00217     {
00218         array[i] = std::move(array[i + 1]);
00219     }
00220
00221     pop_back();
00222
00223     return begin() + idx;
00224 }
00225
00226 // ranges --->
00227 template <typename T>
00228 typename Vector<T>::iterator
00229 Vector<T>::erase(iterator first, iterator last)
00230 {
00231     size_type count = last - first;
00232     size_type idx = first - begin();
00233
00234     // Do nothing
00235     if (count == 0)
00236     {
00237         return end();
00238     }
00239
00240     // Shifting elements to the left
00241     for (size_type i = idx; i + count < size_; ++i)
00242     {
00243         array[i] = std::move(array[i + count]);
00244     }
00245
00246     // Destroying elements
00247     for (size_type d = 0; d < count; ++d)
00248     {
00249         array[--size_].~T();
00250     }
00251
00252     return begin() + idx;
00253 }
00254
00255 template <typename T>
00256 typename Vector<T>::iterator
00257 Vector<T>::erase(const_iterator first, const_iterator last)
00258 {
00259     size_type count = last - first;
00260     size_type idx = first - begin();
00261
00262     // Do nothing
00263     if (count == 0)
00264     {
00265         return end();
00266     }
00267
00268     // Shifting elements to the left
00269     for (size_type i = idx; i + count < size_; ++i)
00270     {
00271         array[i] = std::move(array[i + count]);
00272     }
00273
00274     // Destroying elements
00275     for (size_type d = 0; d < count; ++d)
00276     {

```



```

00277         array[--size_].~T();
00278     }
00279
00280     return begin() + idx;
00281 }
00282
00283 // push_back
00284 // Appends a copy of the given element to the end of the vector.
00285 // (public member function)
00286 template <typename T>
00287 void Vector<T>::push_back(const T &value){
00288     if (size_ == capacity_){
00289         Vector::double_the_capacity();
00290     }
00291     new (&array[size_]) T(value);          // construct from const T&
00292     ++size_;
00293 }
00294
00295 template <typename T>
00296 void Vector<T>::push_back(T &&value)
00297 {
00298     if (size_ == capacity_){
00299         Vector::double_the_capacity();
00300     }
00301     new (&array[size_]) T(std::move(value)); // construct from T&&
00302     ++size_;
00303 }
00304
00305 // emplace_back (C++11)
00306 // Constructs an element in-place at the end of the vector.
00307 // (public member function)
00308 template<typename T>
00309 template< class... Args >
00310 typename Vector<T>::reference Vector<T>::emplace_back(Args&&... args){
00311     auto it = emplace(end(), std::forward<Args>(args)...);
00312     return *it;
00313 }
00314
00315 // pop_back
00316 // Removes the last element from the vector.
00317 // (public member function)
00318 template <typename T>
00319 void Vector<T>::pop_back(){
00320     array[size_ - 1].~T();
00321     --size_;
00322 }
00323
00324 // resize
00325 // Changes the number of elements stored in the vector.
00326 // If new size > old size, value-initializes new elements.
00327 // If new size < old size, erases excess elements.
00328 // (public member function)
00329 template <typename T>
00330 void Vector<T>::resize(size_type count)
00331 {
00332     if (count == size_)
00333     {
00334         return;
00335     }
00336
00337     else if (size_ > count)
00338     {
00339         for (size_type i = count; i < size_; ++i)
00340         {
00341             array[i].~T();
00342         }
00343         size_ = count;
00344         return;
00345     }
00346
00347     else
00348     {
00349         reserve(count);
00350         for (size_type i = size_; i < count; ++i)
00351         {
00352             new (&array[i]) T();
00353         }
00354         size_ = count;
00355         return;
00356     }
00357 };
00358
00359 template <typename T>
00360 void Vector<T>::resize(size_type count, const T &value)
00361 {
00362     if (count == size_)
00363     {

```

```

00364         return;
00365     }
00366
00367     else if (size_ > count)
00368     {
00369         for (size_type i = count; i < size_; ++i)
00370         {
00371             array[i].~T();
00372         }
00373         size_ = count;
00374         return;
00375     }
00376
00377     else
00378     {
00379         reserve(count);
00380         for (size_type i = size_; i < count; ++i)
00381         {
00382             new (&array[i]) T(value);
00383         }
00384         size_ = count;
00385         return;
00386     }
00387 };
00388
00389 // swap
00390 //   Exchanges the contents of this vector with another.
00391 //   (public member function)
00392 template <typename T>
00393 void Vector<T>::swap(Vector &other) noexcept{
00394     std::swap(array, other.array);
00395     std::swap(size_, other.size_);
00396     std::swap(capacity_, other.capacity_);
00397 }

```

4.15 vector_non_member_functions.h File Reference

```
#include "vector_library.h"
```

Functions

- `template<typename T>`
`std::ostream & operator<< (std::ostream &os, const Vector< T > &vec)`

4.15.1 Function Documentation

4.15.1.1 operator<<()

```

template<typename T>
std::ostream & operator<< (
    std::ostream & os,
    const Vector< T > & vec)

```

4.16 vector_non_member_functions.h

[Go to the documentation of this file.](#)

```

00001 // vector_non_member_functions.h
00002 #pragma once
00003
00004 #include "vector_library.h"
00005

```

```

00006 template <typename T>
00007 bool Vector<T>::operator==(const Vector& other) const {
00008     if (size_ != other.size_) return false;
00009
00010     return std::equal(begin(), end(),
00011                     other.begin(), other.end());
00012 }
00013
00014 template <typename T>
00015 bool Vector<T>::operator!=(const Vector& other) const {
00016     return !(other == *this);
00017 }
00018
00019 template <typename T>
00020 bool Vector<T>::operator>(const Vector& other) const {
00021     // true if the first range is lexicographically less than the second, otherwise false.
00022     // Note: always uses < (so it's - other < this)
00023     return std::lexicographical_compare(other.begin(), other.end(), // first range (other)
00024                                         begin(), end());           // second range (*this)
00025 }
00026
00027 template <typename T>
00028 bool Vector<T>::operator>=(const Vector& other) const {
00029     return !std::lexicographical_compare(begin(), end(),
00030                                         other.begin(), other.end())
00031 };
00032 }
00033
00034 template <typename T>
00035 bool Vector<T>::operator<(const Vector& other) const {
00036     return std::lexicographical_compare(begin(), end(),
00037                                         other.begin(), other.end());
00038 }
00039
00040 template <typename T>
00041 bool Vector<T>::operator<=(const Vector& other) const {
00042     return !std::lexicographical_compare(other.begin(), other.end(),
00043                                         begin(), end());
00044 }
00045
00046 template<typename T>
00047 std::ostream& operator<<(std::ostream& os, const Vector<T>& vec) {
00048     os << '[';
00049     for (typename Vector<T>::size_type i = 0; i < vec.size(); ++i) {
00050         os << vec[i];
00051         if (i + 1 < vec.size())
00052             os << ", ";
00053     }
00054     os << ']';
00055     return os;
00056 }

```


Index

`~Vector`
 `Vector< T >`, [11](#)

`assign`
 `Vector< T >`, [11](#), [12](#)

`at`
 `Vector< T >`, [12](#)

`back`
 `Vector< T >`, [13](#)

`begin`
 `Vector< T >`, [13](#)

`capacity`
 `Vector< T >`, [13](#)

`cbegin`
 `Vector< T >`, [13](#)

`cend`
 `Vector< T >`, [14](#)

`clear`
 `Vector< T >`, [14](#)

`const_iterator`
 `Vector< T >`, [9](#)

`const_reference`
 `Vector< T >`, [9](#)

`const_reverse_iterator`
 `Vector< T >`, [9](#)

`crbegin`
 `Vector< T >`, [14](#)

`crend`
 `Vector< T >`, [14](#)

`data`
 `Vector< T >`, [14](#)

`emplace`
 `Vector< T >`, [15](#)

`emplace_back`
 `Vector< T >`, [15](#)

`empty`
 `Vector< T >`, [15](#)

`end`
 `Vector< T >`, [16](#)

`erase`
 `Vector< T >`, [16](#)

`front`
 `Vector< T >`, [17](#)

`insert`
 `Vector< T >`, [17](#), [18](#)

`iterator`
 `Vector< T >`, [9](#)

`max_size`
 `Vector< T >`, [18](#)

`operator!=`
 `Vector< T >`, [18](#)

`operator<`
 `Vector< T >`, [18](#)

`operator<<`
 `vector_non_member_functions.h`, [36](#)

`operator<=`
 `Vector< T >`, [18](#)

`operator>`
 `Vector< T >`, [19](#)

`operator>=`
 `Vector< T >`, [19](#)

`operator=`
 `Vector< T >`, [19](#)

`operator==`
 `Vector< T >`, [19](#)

`operator[]`
 `Vector< T >`, [20](#)

`pop_back`
 `Vector< T >`, [20](#)

`push_back`
 `Vector< T >`, [20](#)

`rbegin`
 `Vector< T >`, [20](#), [21](#)

`reference`
 `Vector< T >`, [9](#)

`rend`
 `Vector< T >`, [21](#)

`reserve`
 `Vector< T >`, [21](#)

`resize`
 `Vector< T >`, [21](#)

`reverse_iterator`
 `Vector< T >`, [9](#)

`shrink_to_fit`
 `Vector< T >`, [22](#)

`size`
 `Vector< T >`, [22](#)

`size_type`
 `Vector< T >`, [9](#)

`swap`
 `Vector< T >`, [22](#)

Vector

Vector< T >, 10, 11

Vector< T >, 5

~Vector, 11

assign, 11, 12

at, 12

back, 13

begin, 13

capacity, 13

cbegin, 13

cend, 14

clear, 14

const_iterator, 9

const_reference, 9

const_reverse_iterator, 9

crbegin, 14

crend, 14

data, 14

emplace, 15

emplace_back, 15

empty, 15

end, 16

erase, 16

front, 17

insert, 17, 18

iterator, 9

max_size, 18

operator!=, 18

operator<, 18

operator<=, 18

operator>, 19

operator>=, 19

operator=, 19

operator==, 19

operator[], 20

pop_back, 20

push_back, 20

rbegin, 20, 21

reference, 9

rend, 21

reserve, 21

resize, 21

reverse_iterator, 9

shrink_to_fit, 22

size, 22

size_type, 9

swap, 22

Vector, 10, 11

vector.h, 23

vector_capacity.h, 26

vector_elementAccess.h, 27

Vector_impl.h, 28

vector_library.h, 29

vector_memberFunctions.h, 29

vector_modifiers.h, 31

vector_non_member_functions.h, 36

operator<<, 36